FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

IT0049
(WEB SYSTEM TECHNOLOGIES)

TECHNICAL FORMATIVE ASSESSMENT

5

FROM ZERO TO FOUR PAGES, AGAIN: YOUR FIRST LARAVEL APPLICATION

Student Name / Group Name:	QUEJADA, RAINA KISHAN S.
Members (if Group):	**Name** / **Role** ( ) / ( ) ( ) / ( ) ( ) / ( ) ( ) / ( )
Section:	TW35
Professor:	Mr. Ric Cristian A. Frias

PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Analyze a complex problem and identify and define the computing requirements appropriate to its solution. [PO: B]

COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- Understand the advanced principles of web development using web frameworks and MVC architecture. [CLO: 1]

INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Set up a new Laravel project and run it locally.
- Use Artisan to inspect and generate application files.
- Create Laravel routes, controllers, and Blade views.
- Use a static PHP array as a temporary data source for a listing page.

BACKGROUND INFORMATION

This activity replicates TFA1, but in Laravel. The MVC responsibilities are exactly the same as CodeIgniter's — a route decides which controller method runs, the controller decides what happens, and a view (here, a Blade template) renders the response — only the file names, folder structure, and syntax differ. Focus on recognizing the same responsibilities rather than learning the ideas from scratch.

GRADING SYSTEM / RUBRIC

CRITERIA	Description
Functionality & Requirements Completeness (40 points max)	All required pages and features are present and work as specified in the activity instructions.
Code Structure & Organization (25 points max)	Controllers, models, and views are separated correctly and follow the framework's conventions; code is readable and sensibly named.
Data Handling with Static Arrays (20 points max)	The static PHP array is structured sensibly, used consistently as the page's temporary data source, and looped through correctly in the Blade view.

CRITERIA	Description
Documentation & Submission Quality (15 points max)	The GitHub repository is organized and complete, the README explains how to set up and run the project, and the hosted link is live and matches the submitted code.

LABORATORY ACTIVITY

Rebuild the POS application from TFA1 in Laravel: a landing page (/), an about page (/about), a Customer Accounts page (/customers), and a User Accounts page (/users), with the account pages using a static PHP array as a temporary data source.

1. Create a new Laravel project and confirm it runs locally with the Artisan development server.
2. Define routes for all four pages in routes/web.php.
3. Create a PageController for the landing and about pages, and a CustomerController and UserController for the account pages, each defining a static PHP array of at least 5 sample records.
4. Create Blade views for all four pages, using @foreach to loop through the static array data in the Customer Accounts and User Accounts views.
5. Add navigation links between all four pages and confirm every route works.

SUCCESSFUL COMPOSER PROJECT INSTALLATION OR PHP ARTISAN --VERSION

```
PS C:\Users\kisha\Documents\LaravelStudentLab> php artisan --version
Laravel Framework 12.70.2
```

SUCCESSFUL MIGRATION

```
PS C:\Users\kisha\Documents\LaravelStudentLab> php artisan migrate

  INFO  Running migrations.

  2026_10_31_042654_create_tasks_table ..... 42.37ms DONE
```

SUCCESSFUL SEEDING

```
PS C:\Users\kisha\Documents\LaravelStudentLab> php artisan make:seeder TaskSeeder

  INFO  Seeder [C:\Users\kisha\Documents\LaravelStudentLab\database\seeders\TaskSeeder.php] created successfully.
```

TASK.PHP

app > Models > Task.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Task extends Model
{
    protected $fillable = ['title', 'status'];
}

```

TASKSEEDER.PHP

database > seeders > TaskSeeder.php

```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use App\Models\Task;

class TaskSeeder extends Seeder
{
    public function run(): void
    {
        Task::create(['title' => 'Buy groceries', 'status' => 'pending']);
        Task::create(['title' => 'Submit report', 'status' => 'pending']);
        Task::create(['title' => 'Fix login bug', 'status' => 'done']);
    }
}
```

TASKCONTROLLER.PHP

app > Http > Controllers > TaskController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Task;
use Illuminate\View\View;

class TaskController extends Controller
{
    public function index(): View
    {
        $tasks = Task::all();
        return view('tasks.index', ['tasks' => $tasks]);
    }
}
```

ROUTES/WEB.PHP

routes > web.php

```php
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\TaskController;


Route::get('/', function () {
    return view('welcome');
});
Route::get('/tasks', [TaskController::class, 'index']);

```

TASKS/INDEX.BLADE.PHP.

resources > views > tasks > index.blade.php

```php
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My Tasks</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            max-width: 850px;
            margin: 40px auto;
            padding: 0 20px;
        }
        table {
            width: 100%;
            border-collapse: collapse;
            box-shadow: 0 1px 3px rgba(0,0,0,0.1);
```

```
        }
        th {
            background: #11223e;
            color: white;
            font-weight: 500;
            text-align: left;
            padding: 14px 16px;
        }
        td {
            padding: 14px 16px;
            color: #333;
            font-size: 14px;
        }
        tbody tr:nth-child(even) {
            background: #f4f7fc;
        }
        tbody tr:nth-child(odd) {
            background: #ffffff;
        }
    </style>
</head>
<body>
    <table>
        <thead>
            <tr>
                <th style="width: 10%;">ID</th>
                <th style="width: 65%;">Task Title</th>
                <th style="width: 25%;">Status</th>
            </tr>
        </thead>
        <tbody>
            @forelse ($tasks as $task)
            <tr>
                <td>{{ $task->id }}</td>
                <td>{{ $task->title }}</td>
                <td>{{ ucfirst($task->status) }}</td>
            </tr>
            @empty
            <tr>
                <td colspan="3" style="text-align: center;">No tasks found.</td>
            </tr>
            @endforelse
        </tbody>
    </table>
</body>
</html>
```

1. **What is the role of the migration?**

   - A migration acts as version control for your database. Instead of manually creating tables using database software, a migration defines the structure (columns, tables, and data types) in PHP code. This ensures that any team member can run the same blueprint command and build an identical database structure automatically.

2. **What is the difference between the model and controller?**

   - **The Model (Task):** Manages the data logic and communicates directly with your database table. It handles data validation, structure rules, and retrieves raw records from the database.

   - **The Controller (TaskController):** Acts as the middleman orchestrating the application flow. It intercepts browser requests, tells the model to grab the data, and passes that data along to the user interface.

3. **How does the Blade view receive task records?**

   - The Blade view receives data through variable extraction passed by the controller. In your code, the controller calls Task::all() to pull all the database rows. It then returns the layout file while injecting an associative array payload: return view('tasks.index', ['tasks' => $tasks]);. This makes a custom $tasks variable dynamically accessible within your HTML file loop.

4. **Why do we use a seeder?**

   - We use a seeder to populate the database with sample or default records automatically. Instead of typing test items out by hand through a frontend form every time a database is reset, running a seeder inserts predictable baseline information instantly so developers can test application functionality immediately.

Submission Requirements:

- A link to a GitHub repository containing your raw project files, including your database export or migration and seeder files.
- A link to your hosted, working version of the application.

REFERENCES

- Stauffer, M. (2024). Laravel: Up & Running: A Framework for Building Modern PHP Apps (3rd ed.).
- Laravel Documentation. Routing. https://laravel.com/docs/13.x/routing
- Laravel Documentation. Blade Templates. https://laravel.com/docs/13.x/blade